



HOCHSCHULE OSNABRÜCK
UNIVERSITY OF APPLIED SCIENCES

Dokumentation der Implementierung des Spiels Tic-Tac-Toe auf dem Nucleo-L412KB

Projekt:	Implementierung von Tic-Tac-Toe auf einem Nucleo-Board mit Steuerung über UART und Tastatur sowie Ausgabe über LCD und LED-Ring.
Abgabetermin:	1. April 2026
Modul:	Eingebettete Systeme
Dozent:	Prof. Dr.-Ing. Steffen Greiser, Dieter Jönen
Studiengang:	Wirtschaftsinformatik
Team:	Aleksandr Leongard (1080072) Kevin Mekelburg (1077281)

Inhaltsverzeichnis

1	Hardwaresetup und Konfiguration	1
1.1	Verdrahtung und Pin-Setup	1
1.2	Konfiguration der Peripherie	3
2	Architektur	3
3	Einsatz globaler Variablen	4
4	Programmablauf	4
4.1	Initialisierung und Hauptsteuerung	5
4.2	Spielablauf	6

1 Hardwaresetup und Konfiguration

Dieses Kapitel dokumentiert die physikalische Verschaltung sowie die softwareseitige Initialisierung der Peripherie. Weitere allgemeine Spezifikationen zu den genutzten Hardwarekomponenten und dem Mikrocontroller sind den Veranstaltungsunterlagen zu entnehmen.

1.1 Verdrahtung und Pin-Setup

Die physische Zuordnung der STM32-Peripheriefunktionen zu den verwendeten Hardwarekomponenten ist entscheidend für die spätere softwareseitige Initialisierung. Die folgende Tabelle fasst alle Verbindungen zusammen. Dabei wird der interne Port-Pin Name sowie der auf der Nucleo-32 Platine aufgedruckte Bezeichner angegeben.

MCU Pin (Nucleo Label)	HAL / Peripheriefunktion	Komponente und Bemerkung
PA2 (VCP_TX)	USART2_TX (Asynchron)	UART (Serielle Ausgabe an PC)
PA15 (VCP_RX)	USART2_RX (Asynchron)	UART (Serielle Eingabe vom PC)
PA9 (A4)	I2C1_SCL	Midas I2C-LCD 2x8 (Takt)
PA10 (A5)	I2C1_SDA	Midas I2C-LCD 2x8 (Daten)
PB4 (D12)	SPI1_MISO (Receive-Only Master)	Keypad Touch-Sensor (Daten / SDO)
PA1 (A1)	SPI1_SCK (Receive-Only Master)	Keypad Touch-Sensor (Takt / SCL)
PA0 (A0)	TIM2_CH1 (PWM via DMA)	LED-Ring (WS2812B Datenleitung)
PB3 (LD3)	GPIO_Output	On-Board Nutzer-LED (LD3)
+3V3 (+3V3)	VCC	Stromversorgung LCD, Keypad, LED-Ring
GND (GND)	GND	Gemeinsames Bezugspotenzial

Tabelle 1: Verdrahtung und Pin-Belegung der Hardwarekomponenten.

Die physikalische Umsetzung auf dem Steckbrett und die Verlegung der Jumper-Wire Leitungen sind in Abbildung 1 dargestellt.

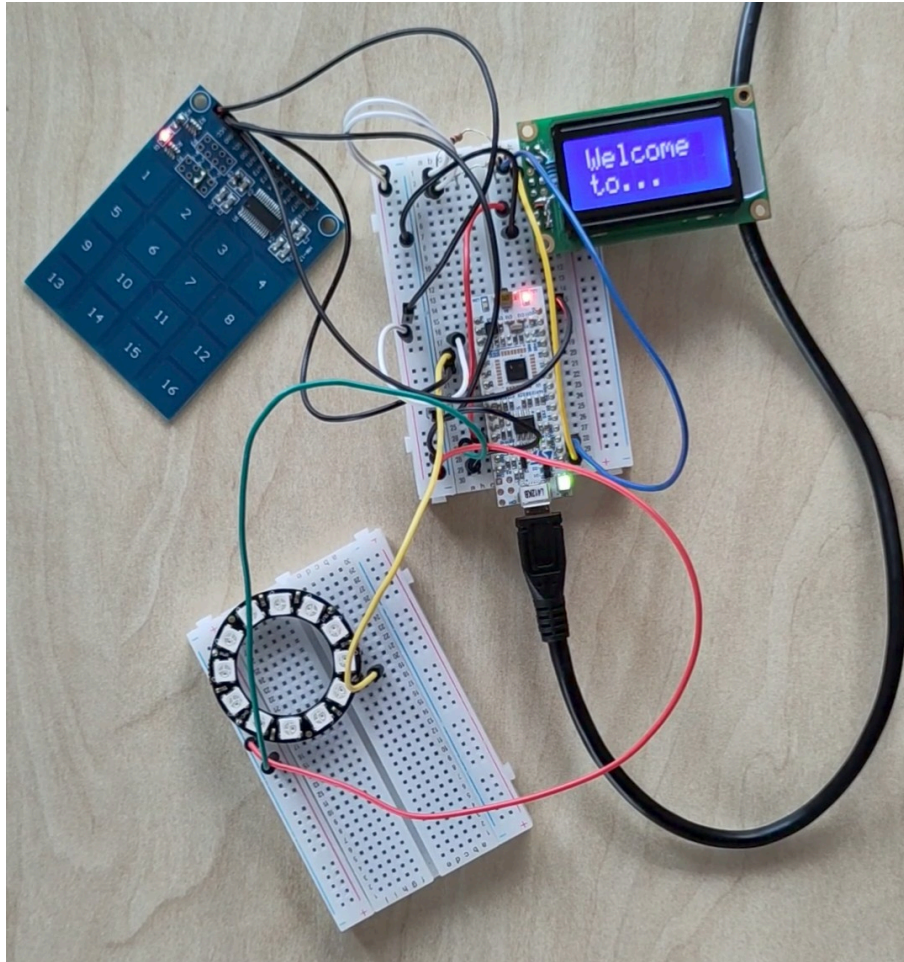


Abbildung 1: Foto des physischen Aufbaus am Nucleo-Board auf einem Steckbrett.

Zum besseren Verständnis der softwareseitigen Peripheriekonfiguration ist in Abbildung 2 zusätzlich der Pinout-View der STM32CubeIDE dargestellt.

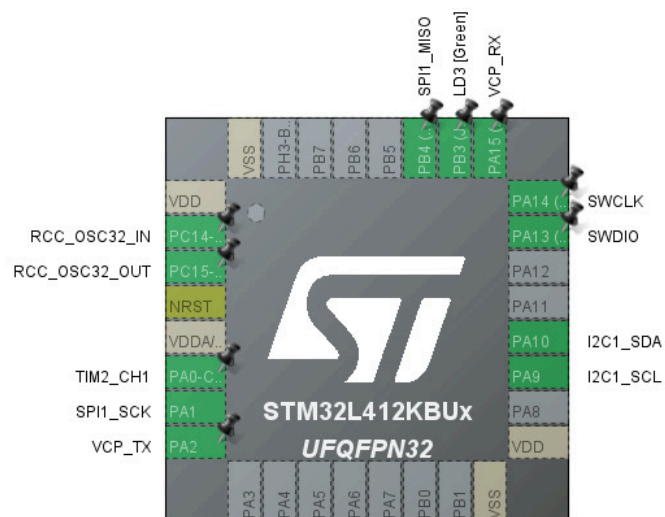


Abbildung 2: Screenshot der Pinout-Konfiguration in der .ioc-Datei des Projekts.

1.2 Konfiguration der Peripherie

Die Initialisierung und hardwarenahe Steuerung erfolgt über das HAL-Framework von STMicroelectronics. Aus der Projektkonfiguration ergeben sich folgende Parameter für die Peripherie-Module:

- **Systemtakt:** Als primäre Taktquelle ist der interne Multispeed-Oszillator mit einer Grundfrequenz von 48 MHz konfiguriert. Über den AHB-Prescaler werden der Systemtakt, der Prozessorkern sowie der APB1-Peripheriebus mit 24 MHz getaktet. Der APB2-Peripheriebus ist über einen Vorteiler auf 1,5 MHz konfiguriert.
- **UART:** USART2 arbeitet im asynchronen Modus und fungiert als Virtual COM Port für die textbasierte PC-Kommunikation.
- **I2C:** I2C1 ist als Standard-I2C-Master konfiguriert, um das Midas-LCD über eine 7-Bit-Adressierung anzusteuern.
- **SPI:** Um das synchrone serielle Protokoll des kapazitiven Touch-Keypads auszulesen, ist SPI1 als Receive-Only Master konfiguriert. Die Taktpolarität steht auf HIGH, die Taktphase auf der zweiten Flanke. Der Vorteiler ist auf 256 gesetzt.
- **Timer & DMA:** Zur Generierung des streng getakteten WS2812B-Datenprotokolls wird Timer 2 im PWM-Modus verwendet. Bei einem Timer-Takt von 24 MHz und einem Auto-Reload-Wert von 29 wird die geforderte Frequenz von 800 kHz erreicht. Die Bitmuster werden blockierungsfrei über den DMA-Controller in das Timer-Register übertragen.
- **Zufallsgenerator (RNG):** Die Hardware-RNG-Peripherie ist aktiviert, um echten Zufall für die Anfangszug bereitzustellen.

2 Architektur

Die Software des Tic-Tac-Toe-Spiels ist nach einem modularen Schichtenmodell aufgebaut. Die physische Dateistruktur des Projekts spiegelt diese Architektur wider, indem die Quellcode-Dateien in Verzeichnisse unterteilt sind. Dies sorgt für eine funktionale Trennung von Hardware-Zugriffen, Spielregeln und der Ein- und Ausgabe.

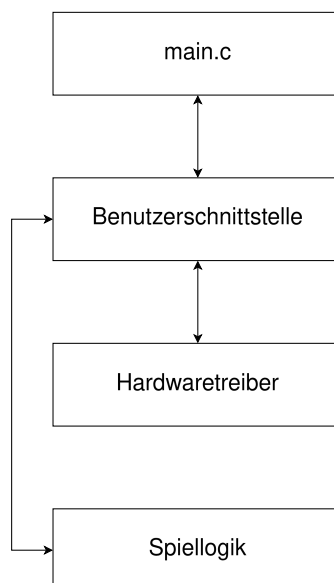


Abbildung 3: Schichtenmodell und Datenfluss der Softwarearchitektur.

Die Architektur gliedert sich, wie in Abbildung 3 dargestellt, in vier grundlegende Komponenten:

Die **main.c** ist der zentrale Einstiegspunkt des Programms. Nach dem Systemstart und der Initialisierung der Hardware-Peripherie wird hier die Hauptschleife gestartet, welche die Kontrollfunktionen der Benutzerschnittstelle aufruft.

Die Schicht **Benutzerschnittstelle** bildet das Bindeglied zwischen dem Benutzer, der Hardware und den Spielregeln. Sie verarbeitet Eingaben und formatiert die Ausgaben, unabhängig ob in UART-Konsole oder auf der Hardware. Um diese Peripherie anzusprechen, kommuniziert sie bidirektional mit den Hardwaretreibern. Gleichzeitig leitet sie Nutzeraktionen an die Spiellogik weiter und fragt den aktuellen Spielstatus ab.

Die **Hardwaretreiber**-Schicht kapselt den direkten Zugriff auf die Elektronik des Mikrocontrollers. Sie enthält keine spielspezifischen Informationen, sondern stellt der Benutzerschnittstelle universelle Funktionen zum Senden und Empfangen von elektrischen Signalen zur Verfügung.

In der Schicht **Spiellogik** ist das reine Regelwerk von Tic-Tac-Toe implementiert. Diese Schicht arbeitet hardwareunabhängig. Sie empfängt Daten von der Benutzerschnittstelle, ändert den internen Spielzustand und gibt Ergebnisse zurück, ohne selbst mit den Treibern zu interagieren.

3 Einsatz globaler Variablen

Globale Variablen werden grundsätzlich vermieden, da sie die Kopplung erhöhen und den Programmzustand schwerer nachvollziehbar machen.

In diesem System werden sie jedoch gezielt eingesetzt, wenn mehrere Funktionen auf denselben Hardwarezustand zugreifen müssen und eine Übergabe über Parameter unpraktisch oder unnötig wäre.

Ein Beispiel ist das SPI-Handle im Tastaturtreiber, das nach der Initialisierung dauerhaft benötigt wird. Ebenso wird der zuletzt erkannte Tastendruck global gespeichert, da die Erfassung und das Auslesen zeitlich getrennt erfolgen.

Im Neopixel-Treiber werden die LED-Zustände und der DMA-Puffer global gehalten, da diese Daten während der gesamten Übertragung verfügbar bleiben müssen.

Die Verwendung ist damit auf wenige, hardwarebezogene Fälle beschränkt. Die globalen Variablen werden nicht zur allgemeinen Datenhaltung des Programms verwendet, sondern gezielt für gemeinsam genutzte Hardwarezustände und persistente Treiberdaten.

4 Programmablauf

Der softwareseitige Ablauf des Systems ist, analog zur Architektur, hierarchisch gegliedert. Die Steuerung unterteilt sich in die grundlegende Systeminitialisierung auf oberster Ebene (**main.c**) und die eigentliche Spielschleife, welche die Spielregeln und die Rundenverwaltung (**Run Game**) kapselt.

4.1 Initialisierung und Hauptsteuerung

Der prinzipielle Lebenszyklus der Anwendung wird durch den in Abbildung 4 dargestellten Programmablaufplan definiert.

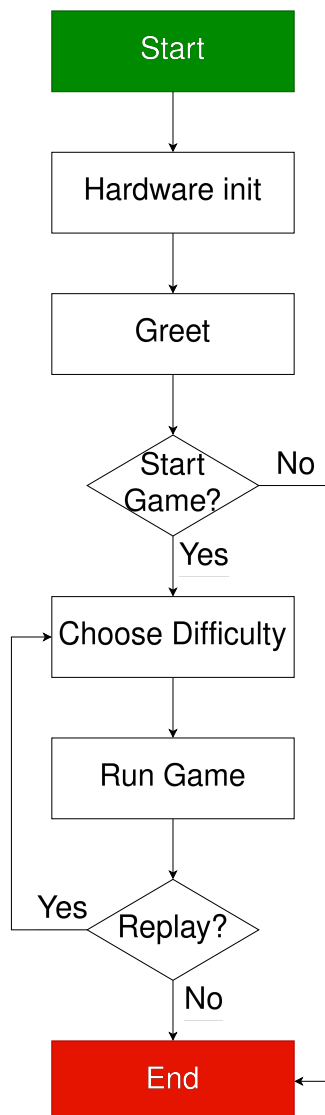


Abbildung 4: Programmablaufplan der Hauptschleife (High-Level-Perspektive).

Nach dem Start des Mikrocontrollers erfolgt zunächst die zwingend notwendige Initialisierung der Hardware. In dieser Phase werden die Systemtakte konfiguriert und die verwendeten Peripheralschnittstellen über die entsprechenden HAL-Funktionen parametrisiert und betriebsbereit gemacht.

Sobald die Hardware einsatzbereit ist, wird der Benutzer über die verfügbaren Ausgabeschnittstellen, also LCD und UART-Terminal, begrüßt. Im Anschluss wartet das System auf die Bestätigung des Benutzers, ein neues Spiel zu beginnen.

Wird der Startbefehl verweigert, wird das Programm vorzeitig beendet. Bestätigt der Nutzer den Start, wird er zur Auswahl des Schwierigkeitsgrades aufgefordert. Diese Eingabe bestimmt das Verhalten der KI in der nachfolgenden Spielphase.

Daraufhin übergibt die Hauptsteuerung die Kontrolle an die eigentliche Spielschleife, in der das Tic-Tac-Toe-Match ausgetragen wird. Erst wenn diese Funktion durch einen Sieg oder ein Unentschieden beendet wird, kehrt der Programmfluss in die Hauptschleife zurück. Dort wird dem Benutzer angeboten, eine weitere Runde zu spielen. Stimmt er zu, beginnt der Zyklus erneut bei der Auswahl des Schwierigkeitsgrades. Lehnt er ab, endet der Programmablauf.

4.2 Spielablauf

Die Kernlogik des Tic-Tac-Toe-Spiels ist in der Funktion zur Rundenverwaltung gekapselt. Der detaillierte Ablauf eines einzelnen Matches ist im Programmablaufplan in Abbildung 5 visualisiert.

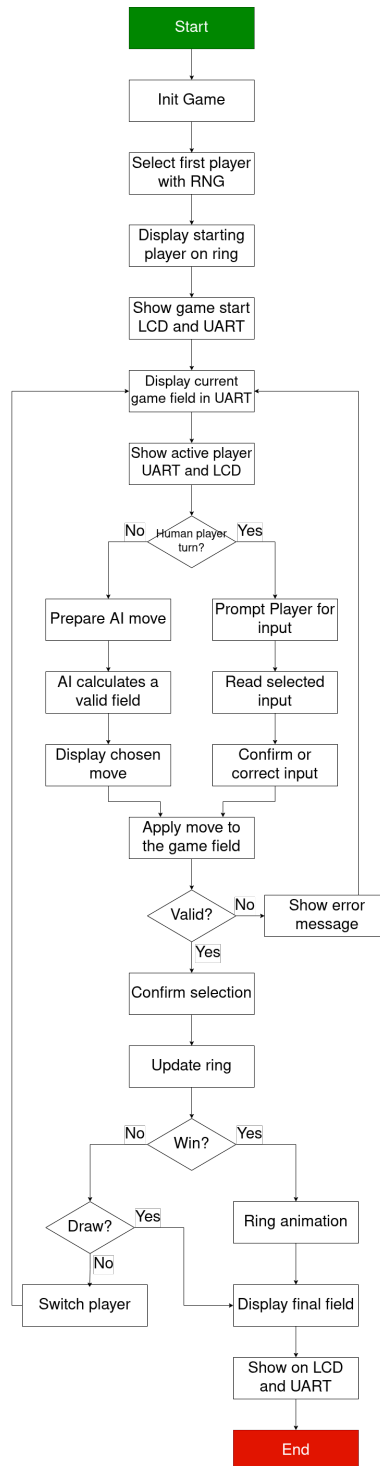


Abbildung 5: Programmablaufplan der Kernlogik und Rundenverwaltung

Das Spiel beginnt mit der Initialisierung der internen Datenstrukturen, insbesondere dem Leeren des 3x3-Spielfeldes. Um einen fairen Spielstart zu gewährleisten, ermittelt der Zufallsgenerator den beginnenden Spieler. Dieser Startspieler wird optisch auf dem LED-Ring signalisiert, gefolgt von einer initialen Startmeldung auf dem LCD und über die UART-Schnittstelle.

Anschließend geht das Programm in die zyklische Spielschleife über. Zu Beginn jedes Durchlaufs wird das aktuelle Spielfeld im UART-Terminal gezeichnet und der aktive

Spieler auf dem LCD sowie via UART angezeigt. Der Kontrollfluss verzweigt sich danach abhängig davon, ob ein menschlicher Spieler oder die KI am Zug ist.

Bei dem menschlichen Spieler fordert das System den Benutzer zur Eingabe auf. Die Eingabe über die Tastatur oder die Peripherie wird eingelesen und dem Spieler zur Überprüfung oder Korrektur angeboten.

Für den KI-Spieler wird die Zugberechnung vorbereitet. Die KI ermittelt basierend auf dem gewählten Schwierigkeitsgrad ein gültiges und strategisch sinnvolles Feld und zeigt den gewählten Zug an.

Der ermittelte Zug wird anschließend auf das Spielfeld angewendet. Es folgt eine Validierungsprüfung. Ist der Zug ungültig, z. B. ist das Feld bereits belegt oder die Eingabe fehlerhaft, wird eine Fehlermeldung ausgegeben und die Schleife springt zurück, sodass derselbe Spieler seinen Zug wiederholen muss.

Ist der Zug gültig, wird die Auswahl bestätigt und der LED-Ring entsprechend aktualisiert. Im nächsten Schritt wertet die Spiellogik die neue Feldbelegung aus:

1. **Siegbedingung:** Hat der aktuelle Zug zu einer vollständigen Reihe geführt, wird eine Siegesanimation auf dem LED-Ring abgespielt. Das finale Spielfeld wird zusammen mit der Siegesmeldung auf LCD und UART ausgegeben. Die Spielschleife endet.
2. **Unentschieden:** Ist das Feld vollständig gefüllt, ohne dass eine Siegbedingung erfüllt ist, wird das finale Spielfeld angezeigt und das Spiel endet mit einem Unentschieden.
3. **Weiterspielen:** Tritt weder Sieg noch Unentschieden ein, wird der aktive Spieler gewechselt und die Schleife beginnt von vorn mit der Anzeige des aktualisierten Spielfeldes.